Tab 1

AI-Powered QA Automation Scale-Up

1. Executive Summary

Traditional QA automation is failing to scale across thousands of multi-tenant configurations and frequent AI-model updates. This PRD outlines an **AI-Native QA Engine** that utilizes self-healing scripts, synthetic data generation, and "Agentic" testing to ensure 99.9% platform reliability for enterprise clients.

2. Problem Statement

- **Brittle Suites:** Frequent UI updates in the Gateway and Workpapers cause high maintenance overhead for traditional Selenium/Cypress scripts.
- **Data Silos:** Testing complex tax logic (1040s, 1120s) requires diverse, anonymized datasets that are currently difficult to generate at scale.
- **AI Blind Spots:** Existing QA does not validate the "intelligence" of AI agents (e.g., if a document is summarized correctly or a tax line is mapped accurately).

3. Goals & Strategic Objectives

- **Scalability:** Reduce QA cycle time by 70% through parallel AI-driven execution.
- **Self-Healing:** Implement computer-vision-based testing that adapts to UI changes without manual code updates.
- **Cognify Integration:** Provide an API-first QA layer that Cognify clients can integrate into their own CI/CD pipelines to validate their custom modules.

---

4. Functional Requirements

4.1. Self-Healing AI Test Controller

- **Requirement:** The system must automatically detect UI changes (e.g., a button moving from "Engage" to "Gather") and update test selectors in real-time.
- **Tech:** Use LLM-based reasoning to interpret intended actions rather than hardcoded XPath/CSS selectors.

4.2. Synthetic Tax-Data Generator

- **Requirement:** An AI module to generate thousands of unique, compliant, but "fake" tax profiles (W2s, K-1s) to test the **Tax Organizer & PBC** module.
- **Constraint:** Must adhere to HubSync's strict Privacy & Security standards, ensuring no PII is ever used in testing.

4.3. "Agent-on-Agent" Testing

- **Requirement:** Deploy specialized QA Agents to test HubSync's production AI Agents.
    - *Validation:* Does the AI Assistant correctly interpret a complex 1120S form?

- *Regression:* Does a model update in HubSync AI cause accuracy degradation in document classification?

4.4. Multi-Tenant Scalability (Cognify-Specific)

- **Requirement:** A dashboard for Cognify clients to view QA health across their specific custom "Tiles" and integrations.
- **Feature:** "One-Click Performance Test" for high-volume tax seasons to simulate 10,000+ simultaneous document uploads.

---

5. User Experience (UX) for QA Engineers

- **Natural Language Test Creation:** QA engineers should be able to write tests in plain English: *"Upload a 1040 PDF, verify the AI Assistant extracts the AGI correctly, and then trigger an e-Signature."*
- **Visual Debugging:** A "Heat Map" in HubSync Analytics showing where AI confidence is low during automated test runs.

6. Technical Constraints & Security

- **Model Context Protocol (MCP):** The QA framework must use MCP standards to connect testing agents with internal databases and third-party tools (Teams, Outlook).
- **Cloud Native:** Must run in parallel containers (Docker/K8s) to scale resource usage during peak QA cycles.

---

7. Success Metrics

Metric	Current State	Target (Post-AI QA)
Test Maintenance Time	20 hours/week	< 2 hours/week
Release Velocity	Bi-weekly	Daily / Continuous
Bug Leakage to Production	5%	< 0.5%
AI Mapping Accuracy	92%	99%+ (via continuous AI-QA feedback)

Tab 2

1. Proposed Technical Architecture

The system is divided into three layers to ensure isolation, scalability, and AI reasoning:

Layer	Component	Function
Orchestration	**AI Test Controller**	Interprets natural language and maps it to HubSync's API & Integration endpoints.
Execution	**Vision-Based Runners**	Uses computer vision (not just DOM selectors) to interact with the Gateway and Workpapers.
Intelligence	**The "Winnie" Feedback Loop**	A specialized LLM agent (named after HubSync's mascot) that analyzes failure logs to distinguish between a "UI change" and a "true bug."

2. Sample "Natural Language" Test Script

For a Cognify client integrating the **Tax Organizer**, a test script no longer looks like code. It looks like a business requirement:

Test Scenario: Automated Source Data Extraction

1. **Given:** A new client is onboarded to the Gateway.
2. **Action:** Upload a sample "2025 Form 1040" to the Tax Organizer & PBC module.
3. **AI Validation:** Ensure the AI Assistant correctly identifies the taxpayer's name and Social Security Number.
4. **Integration Check:** Verify the data flows into the Tax Workpapers without manual entry.
5. **Success Criteria:** Extraction accuracy must be $>98\%$ and the status in Analytics must update to "Review Ready."

3. High-Level System Flow

1. **Ingestion:** The QA Engineer types the scenario above into the HubSync QA Portal.
2. **Synthesis:** The **AI Controller** generates a temporary execution plan, spinning up headless browser instances in parallel to simulate scale.
3. **Self-Healing:** If the "Upload" button has been moved to a new sidebar in the latest product update, the **Vision Runner** identifies the button by its icon/context rather than failing on a broken CSS link.
4. **Reporting:** Results are pushed to a dedicated Analytics dashboard for the client, showing real-time ROI on their automation.

AI Self-Healing Decision Logic

When a test step fails to find an element (e.g., the "Submit" button), the system triggers the following evaluation:

Step 1: Contextual Re-Scan

Instead of immediate failure, the **Vision Runner** takes a DOM snapshot and a screenshot. It asks the LLM:

> *"I was looking for 'Submit' at XPath_Alpha. I don't see it. Do you see any element on this page that performs the 'Submit' intent for a Tax Return?"*

Step 2: The Decision Matrix

The AI evaluates the discrepancy based on the following table:

Observation	AI Inference	Action
Element exists but moved from Top Nav to a Sidebar.	**UI Refactor**	**Auto-Heal:** Update the test's element map, log the change, and continue execution.
Element exists but the label changed from "Send" to "File e-Sign."	**Copy Update**	**Auto-Heal:** Update the "Intent Map" and notify the QA lead of the terminology change.

Element is missing entirely, and no similar "Intent" is found.	**Functional Gap**	**Fail:** Mark as a "Defect," capture a video of the failure, and alert the engineering team.
Element is present but clicking it results in a 500 Error.	**System Bug**	**Fail:** Categorize as "Backend Regression" and automatically pull logs from the Gateway.

Step 3: Verification & Human-in-the-Loop

- **Shadow Mode:** For the first 5 instances of an "Auto-Heal," the system flags the change for a QA Engineer to "Approve" or "Reject" in the Analytics dashboard.
- **Permanent Update:** Once approved, the underlying test script is updated automatically for all future runs, ensuring the technical debt of "fixing scripts" is eliminated.

Tab 3

Product: AI-Native Quality Engineering Platform (QAI Platform)

1. Executive Summary

The AI-Native Quality Engineering Platform (QAI Platform) is an enterprise-grade system designed to transform software quality from a reactive, manual, and automation-heavy function into a proactive, autonomous, and intelligence-driven capability.

Unlike traditional QA tools that focus on script execution and automation coverage, this platform introduces a **closed-loop, AI-orchestrated quality system** that continuously learns from code, tests, defects, and production signals to optimize quality outcomes.

The platform enables organizations to:

- Move from test execution → **quality intelligence**
- Move from automation → **autonomous assurance systems**
- Move from reactive QA → **predictive, risk-driven quality engineering**

2. Problem Statement

Modern enterprises face systemic limitations in scaling quality engineering:

2.1 Structural Challenges

- Exponential growth in test cases due to microservices and distributed architectures
- Fragmented QA tooling landscape (UI, API, performance, data, security)
- Lack of integration between development, QA, and production insights

2.2 Operational Challenges

- High maintenance overhead for automation frameworks
- Regression cycles becoming bottlenecks in release velocity
- Limited visibility into release readiness and risk

2.3 Intelligence Gaps

- No unified system to determine *what should be tested*

- Lack of predictive insights (defect likelihood, risk hotspots)
- QA systems do not learn from historical or production data

3. Objectives

3.1 Business Objectives

- Reduce regression cycle time by 60–80%
- Improve release frequency by 2–3x
- Reduce QA operational costs by 30–50%
- Improve release confidence and reduce escaped defects

3.2 Technical Objectives

- Enable AI-driven test generation and optimization
- Implement risk-based, intelligent test selection
- Establish a unified quality intelligence layer
- Enable autonomous QA workflows via agentic orchestration

4. Product Vision

To build a **self-learning, AI-native quality platform** that continuously orchestrates, optimizes, and executes quality processes across the SDLC.

The platform will act as a **Quality Operating System (Quality OS)** for the enterprise.

5. Core Differentiators

5.1 Code-Aware Quality Intelligence

Understands code changes, dependencies, and system architecture to drive testing decisions.

5.2 Agentic Orchestration Layer

AI agents autonomously plan, generate, execute, and optimize testing workflows.

5.3 Closed-Loop Learning System

Continuously learns from:

- Defects
- Test outcomes
- Production telemetry

5.4 Risk-Based Testing Engine

Prioritizes testing based on business impact and failure probability.

5.5 Unified Quality Command Center

Provides real-time, enterprise-wide visibility into quality, risk, and release readiness.

6. Target Users

- QA Engineers / SDETs
- Software Developers
- DevOps Engineers
- Product Managers
- Engineering Leadership
- CIO / CTO Organizations

7. System Architecture

7.1 Architectural Principles

- AI-first, not automation-first
- Modular and extensible
- Cloud-native and horizontally scalable
- Event-driven architecture
- API-first design

7.2 Core Layers

1. Ingestion Layer

- Code repositories (Git)
- API contracts (OpenAPI/Swagger)
- Test artifacts
- Execution logs
- Production telemetry

2. Intelligence Layer

- Code graph engine
- Dependency mapping engine
- Risk scoring models
- Defect prediction models

3. AI/LLM Layer

- Multi-model orchestration (OpenAI, Claude, OSS models)
- Prompt orchestration framework
- RAG-based contextual understanding

4. Agentic Orchestration Layer

- Autonomous agents coordinating workflows
- Task decomposition and execution

5. Execution Layer

- Test execution engines
- CI/CD integration
- Distributed execution (Kubernetes)

6. Experience Layer

- Dashboards
- Developer/QA interfaces
- APIs

8. Functional Requirements

8.1 Code Intelligence Engine

Capabilities:

- Parse code repositories and commits
- Build dependency graphs across services
- Identify impacted components

Outputs:

- Change impact map

- Risk scoring per component

8.2 AI Test Generation Engine

Capabilities:

- Generate test cases from requirements, APIs, and code
- Convert to executable scripts
- Maintain test versioning

Outputs:

- UI, API, and data test scripts

8.3 Intelligent Regression Engine

Capabilities:

- Select minimal viable test suite
- Prioritize tests based on risk

Outputs:

- Optimized regression suite

8.4 Self-Healing Automation

Capabilities:

- Auto-fix broken scripts
- Detect flaky tests

8.5 Agentic QA Orchestration

Agents:

- Planning Agent
- Generation Agent
- Execution Agent
- Analysis Agent

Capabilities:

- Autonomous workflow execution
- Continuous optimization

8.6 Test Data Intelligence

Capabilities:

- Synthetic data generation
- Data masking and compliance
- Scenario-based data creation

8.7 Execution Engine

Capabilities:

- Parallel execution
- CI/CD integration
- Cloud scaling

8.8 Quality Command Center

Capabilities:

- Release readiness scoring
- Risk heatmaps
- Defect prediction
- Coverage analytics

9. Non-Functional Requirements

9.1 Scalability

- Support enterprise-scale workloads (10k–100k test cases)

9.2 Performance

- Real-time or near real-time analysis

9.3 Security

- SOC2, GDPR, enterprise compliance

9.4 Reliability

- 99.9% uptime SLA

10. Data Architecture

Data Sources:

- Code repositories
- Test artifacts
- Execution logs
- Defect tracking systems
- Production monitoring systems

Data Processing:

- Event-driven pipelines
- Batch + streaming processing

11. Integration Requirements

- GitHub, GitLab, Bitbucket
- CI/CD tools (Jenkins, GitHub Actions)
- Test frameworks
- Cloud platforms (AWS, Azure, GCP)
- Observability tools (Datadog, Splunk)

12. AI/ML & LLM Design

Capabilities:

- Test generation via LLMs
- Risk prediction via ML models
- RAG-based contextual reasoning

Design Principles:

- Model-agnostic architecture
- Continuous learning pipelines
- Human-in-the-loop validation

13. Success Metrics

- Reduction in regression time
- Increase in automation efficiency
- Reduction in escaped defects
- Improvement in release confidence

14. Roadmap

The product will evolve through iterative capability expansion across the following dimensions:

- AI-driven test generation and automation expansion
- Intelligent regression and risk-based optimization
- Self-healing and autonomous maintenance capabilities
- Agentic orchestration across the QA lifecycle
- Predictive quality analytics and enterprise command center maturity

This evolution will be driven by business priorities, adoption maturity, and continuous feedback loops rather than fixed phases.

15. Risks & Mitigation

- Adoption resistance → Training programs
- Data quality issues → Validation pipelines
- Model inaccuracies → Feedback loops

16. Future Enhancements

- Autonomous defect resolution
- AI testing for LLM systems
- Cross-product intelligence layer

17. Conclusion

The AI-Native Quality Engineering Platform represents a paradigm shift from automation-centric QA to intelligent, autonomous, and scalable quality systems, enabling enterprises to achieve faster releases, higher confidence, and lower operational overhead.